Bidirectional Backpropagation Neural Machine Translation

Experiments v2 and v3: BPE Tokenization, Scaled Dot-Product Attention, Stacked Encoder, and Cycle Consistency Warmup

March 13, 2026

Contents

1 Overview and Context **3**
- 1.1 Experiment History . . . 3

2 Experiment v2: BPE Tokenization + Scaled Dot-Product Attention **3**
- 2.1 Motivation: Problems Diagnosed in v1 . . . 3
- 2.2 Architecture Changes . . . 4
  - 2.2.1 Fix 1: BPE Tokenization . . . 4
  - 2.2.2 Fix 2: Scaled Dot-Product Attention . . . 4
  - 2.2.3 Fix 3: Input Feeding (Luong et al., 2015) . . . 4
  - 2.2.4 B-BP Cell: Unchanged . . . 4
  - 2.2.5 Four Inference Modes . . . 4
- 2.3 Model Parameters . . . 5
- 2.4 Training Results . . . 5
- 2.5 Diagnosis: Why v2 Failed . . . 5

3 Experiment v3: Stacked Encoder + Cycle Warmup **6**
- 3.1 Architecture Changes from v2 . . . 6
  - 3.1.1 Fix 1: Higher Learning Rate . . . 6
  - 3.1.2 Fix 2: Cycle Consistency Warmup Schedule . . . 6
  - 3.1.3 Fix 3: 2-Layer Stacked B-BP Encoder . . . 6
- 3.2 Model Parameters . . . 6
- 3.3 Training Results . . . 7
  - 3.3.1 Final Test Results (Best Checkpoint: Epoch 20) . . . 7
- 3.4 Positive Signal: Val Loss Improvement . . . 7
- 3.5 Diagnosis: Why v3 BLEU Remains Zero . . . 8
  - 3.5.1 Decoder Fixed-Point Collapse (Confirmed by Probing) . . . 8
  - 3.5.2 Tied Embedding Lock (Confirmed by Flat Output Distribution) . . . 8

4 Planned Fixes for v4 **8**

5 Cross-Experiment Summary **9**

5.1 Interpretation for the Adigun-Kosko Framework 9

1 Overview and Context

This report documents Experiments v2 and v3 of the Bidirectional Backpropagation (B-BP) Neural Machine Translation project, building on the Adigun-Kosko framework of neural network inversion via weight transposition. The core B-BP claim is:

$$\text{Forward:} \quad h_t = \tanh(h_{t-1}\mathbf{U} + x_t + b) \qquad \text{Backward:} \quad h_t = \tanh\Big(h_{t-1}\mathbf{U}^\top + x_t + b\Big) \tag{1}$$

A single recurrent matrix $\mathbf{U} \in \mathbb{R}^{d\times d}$ governs both directions. The forward pass uses $\mathbf{U}$; the backward (inverted) pass uses $\mathbf{U}^\top$ — a PyTorch view with no additional parameters. If $\mathbf{U}$ is orthogonal, $\mathbf{U}^\top = \mathbf{U}^{-1}$ and the backward path is the true algebraic inverse of the forward path.

1.1 Experiment History

v1 Baseline B-BP NMT: word-level 10k vocab, no attention, single-layer encoder. Achieved EN→DE 5.37 BLEU / DE→EN 3.47 BLEU (test). Diagnosed problems: `<unk>` flooding, decoder collapse to "die `<unk>` `<unk>`...", gradient oscillation between directions.

v2 Added BPE tokenization (8k vocab) and scaled dot-product attention. Result: near-zero BLEU for all 8 epochs observed. Diagnosed: learning rate too low, cycle loss competing too early, single-layer encoder insufficient.

v3 Added 2-layer stacked encoder, cycle warmup schedule, higher learning rate. Result: val loss improved to 12.61, but BLEU remained ≈ 0.01. Diagnosed: output tied embedding lock + decoder input feeding causing fixed-point repetition collapse.

2 Experiment v2: BPE Tokenization + Scaled Dot-Product Attention

2.1 Motivation: Problems Diagnosed in v1

Three structural problems were identified from the v1 results:

1. **`<unk>` flooding.** Word-level 10k vocabulary on WMT14 news text caused heavy out-of-vocabulary rates. Source and target sentences contained long runs of `<unk>` tokens, preventing the decoder from learning meaningful alignment.
2. **Decoder collapse.** Without attention, the entire source sentence must be compressed into a single vector h_T. Under the B-BP inversion constraint, $\mathbf{U}^\top$ must approximately invert $\mathbf{U}$ at every step ($\mathbf{U}^\top\mathbf{U} \approx \mathbf{I}$), placing strong near-orthogonality pressure on $\mathbf{U}$. The tanh nonlinearity is lossy, making lossless inversion through h_T impossible. The decoder collapsed to outputting "die `<unk>` `<unk>`..." on every sentence.
3. **Gradient conflict.** $\mathbf{U}^\top$ is a PyTorch view of $\mathbf{U}$. EN→DE and DE→EN gradients compete over the same 65,536 parameters. The DE→EN BLEU oscillated: 15.94 → 2.63 → 0.12 → 2.63, indicating that each EN→DE gradient update destroyed the DE→EN configuration.

2.2 Architecture Changes

2.2.1 Fix 1: BPE Tokenization

SentencePiece BPE models trained on 200k subsampled WMT14 sentence pairs:

- Vocabulary: 8,000 tokens per language (EN and DE separately)
- Model type: BPE (`model_type="bpe"`)
- Character coverage: 1.0 (full Latin script coverage)
- Special tokens: `<pad>`=0, `<unk>`=1, `<sos>`=2, `<eos>`=3

BPE splits out-of-vocabulary words into known subword pieces, reducing OOV rate to near zero. For example, an unknown compound German word is split into recognizable morphemes.

2.2.2 Fix 2: Scaled Dot-Product Attention

The encoder stores all hidden states $\mathbf{H} = [h_1, \ldots, h_T]$ as a (T, B, d) tensor. The decoder attends over all encoder states at each step:

$$\text{scores}i = \frac{h_{\text{dec}} \cdot h_{\text{enc},i}}{\sqrt{d}} \qquad \text{(no additional learned parameters)} \tag{2}$$

$$\alpha = \text{softmax}(\text{scores}, \text{ masked at PAD positions}) \tag{3}$$

$$\text{context} = \alpha \cdot \mathbf{H} \tag{4}$$

No learned scoring parameters were added, preserving the B-BP spirit of minimal parameterization. PAD positions are masked to -10^9 before softmax.

2.2.3 Fix 3: Input Feeding (Luong et al., 2015)

At each decoder step t, the previous context vector is concatenated to the current token embedding as additional input:

$$x_t = \text{Embed}(y_{t-1}) + \text{context}_{t-1} \tag{5}$$

This gives the decoder memory of what it has previously attended to.

2.2.4 B-BP Cell: Unchanged

The core B-BP recurrent cell is identical to v1:

$$h_t = \tanh(h_{t-1}\,\mathbf{W} + x_t + b)\,, \quad \mathbf{W} = \begin{cases} \mathbf{U} & \text{(bbp=False, forward)} \\ \mathbf{U}^\top & \text{(bbp=True, backward)} \end{cases} \tag{6}$$

2.2.5 Four Inference Modes

Mode	Encode	Decode	Purpose
EN→DE	$\mathbf{U}$	$\mathbf{U}$	Forward translation
DE→EN	$\mathbf{U}^\top$	$\mathbf{U}^\top$	Backward (B-BP inversion)
EN cycle	$\mathbf{U}$	$\mathbf{U}^\top$	EN self-reconstruction
DE cycle	$\mathbf{U}^\top$	$\mathbf{U}$	DE self-reconstruction

2.3 Model Parameters

Component	Parameters	Notes
`cell.U` (recurrent)	65,536	Shared: forward $\mathbf{U}$, backward $\mathbf{U}^\top$
`E_en` + `E_de` (tied)	4,096,000	BPE 8k each, tied to output projection
`ctx_proj` (attention output)	65,536	Only new parameter vs v1
Biases + other	768	
Total	**4,227,840**	

Hyperparameters: `hid=256`, `batch=64`, `lr=3e-4`, $\lambda = 0.3$, `epochs=20`

2.4 Training Results

Setup: WMT14 EN-DE, 131,883 training pairs (after BPE filtering), 2,255 validation pairs (newstest), 2,141 test pairs. GPU: NVIDIA Tesla T4.

Ep	Train	NMT	Cyc	Val	EN→DE	DE→EN	s
1	29.0412	22.2967	22.4818	14.8419	0.02	0.00	675
2	19.0229	14.5750	14.8263	14.3367	0.00	0.01	689
3	18.5210	14.2150	14.3533	14.2493	0.02	0.00	494
4	18.3236	14.0693	14.1813	14.2336	0.01	0.00	492
5	18.2115	13.9881	14.0782	14.2569	0.02	0.01	491
6	18.1223	13.9219	14.0012	14.2022	0.00	0.01	489
7	18.0513	13.8699	13.9381	14.1678	0.00	0.01	490
8	17.9799	13.8152	13.8821	14.1757	0.01	0.02	488

Note: v2 was terminated at epoch 8 after diagnosis confirmed learning failure. Val loss moved only from 14.84 to 14.18 over 8 epochs (4.4% improvement).

2.5 Diagnosis: Why v2 Failed

Three compounding problems were identified:

1. **Learning rate too low.** lr $= 3 \times 10^{-4}$ was insufficient to coordinate the new interacting components (attention weights, input feeding context, and B-BP recurrent dynamics) simultaneously. Gradients were too small to move all parameters meaningfully.

2. **Cycle loss competing with translation.** Applying $\lambda = 0.3$ from epoch 1 caused the cycle consistency loss to compete with the translation signal before the model had learned basic word alignment. The model could not prioritize learning to translate.

3. **Insufficient encoder capacity.** A single-layer tanh encoder over 40-token sequences suffers from vanishing gradients through the tanh chain. The encoder could not build sufficiently rich representations for the attention mechanism to exploit.

3 Experiment v3: Stacked Encoder + Cycle Warmup

3.1 Architecture Changes from v2

3.1.1 Fix 1: Higher Learning Rate

Learning rate increased from 3×10^{-4} to 1×10^{-3}. This provides sufficient gradient signal to move all interacting components simultaneously.

3.1.2 Fix 2: Cycle Consistency Warmup Schedule

Instead of applying a fixed $\lambda = 0.3$ from epoch 1, a warmup schedule introduces the inversion constraint gradually:

$$\lambda(e) = \begin{cases} 0.00 & e \leq 5 \quad \text{(translation only)} \\ 0.05 & 6 \leq e \leq 10 \quad \text{(gentle inversion)} \\ 0.10 & e \geq 11 \quad \text{(full B-BP inversion constraint)} \end{cases} \tag{7}$$

This allows the model to learn basic word alignment and translation in the first 5 epochs before the inversion constraint is introduced.

3.1.3 Fix 3: 2-Layer Stacked B-BP Encoder

The single-layer encoder is replaced with a 2-layer stacked architecture. Both layers share the **same** $\mathbf{U}$ matrix, preserving the single-parameter spirit of the Adigun-Kosko framework:

$$h^{(1)}t = \tanh\Big(h_{t-1}^{(1)}\,\mathbf{W} + \text{Embed}(x_t) + b_{\text{enc1}}\Big) \tag{8}$$

$$h^{(2)}t = \tanh\Big(h_{t-1}^{(2)}\,\mathbf{W} + h^{(1)}t + b_{\text{enc2}}\Big) \tag{9}$$

where $\mathbf{W} = \mathbf{U}$ (forward) or $\mathbf{W} = \mathbf{U}^\top$ (backward). Attention attends over $\mathbf{H}^{(2)}$ (top-layer states). The decoder is initialized from $h_T^{(2)}$.

3.2 Model Parameters

Component	**Parameters**	**Notes**
`cell.U` (recurrent)	65,536	Shared: enc-L1, enc-L2, decoder
`E_en` + `E_de` (tied)	4,096,000	BPE 8k each
`ctx_proj` (attention output)	65,536	
Biases + other	1,024	Two encoder biases vs one in v2
Total	**4,228,096**	+256 params vs v2

Hyperparameters: `hid=256`, `batch=64`, `lr=1e-3`, `enc_layers=2`, `epochs=20`, cycle warmup as above.

3.3 Training Results

Ep	Train	NMT	Cyc	Val	EN→DE	DE→EN	λ	s
1	17.5548	17.5548	17.5203	14.2677	0.00	0.03	0.00	595
2	14.4822	14.4822	14.4921	14.2806	0.01	0.02	0.00	575
3	14.4207	14.4207	14.4401	14.2094	0.01	0.00	0.00	557
4	14.2349	14.2349	14.2707	14.1562	0.00	0.00	0.00	553
5	14.1198	14.1198	14.1731	13.9532	0.02	0.00	0.00	551
6	14.6793	13.9769	14.0473	14.1090	0.01	0.02	0.05	549
7	14.5307	13.8345	13.9245	13.7693	0.00	0.00	0.05	549
8	14.3919	13.7004	13.8319	13.9333	0.00	0.00	0.05	547
9	14.2315	13.5445	13.7415	13.5148	0.01	0.00	0.05	549
10	14.0587	13.3790	13.5938	13.5891	0.01	0.01	0.05	548
11	14.5519	13.2180	13.3389	13.4081	0.01	0.00	0.10	548
12	14.3848	13.0696	13.1523	13.3358	0.01	0.00	0.10	544
13	14.2364	12.9353	13.0107	13.4727	0.01	0.02	0.10	541
14	14.0904	12.8030	12.8740	13.1832	0.01	0.00	0.10	544
15	13.9592	12.6848	12.7448	13.0352	0.01	0.00	0.10	543
16	13.8262	12.5646	12.6156	13.0387	0.01	0.00	0.10	541
17	13.7024	12.4527	12.4971	12.9959	0.01	0.00	0.10	542
18	13.5768	12.3390	12.3775	12.7729	0.01	0.01	0.10	539
19	13.4723	12.2441	12.2828	12.7748	0.01	0.00	0.10	538
20	13.3691	12.1509	12.1821	12.6141	0.01	0.00	0.10	540

3.3.1 Final Test Results (Best Checkpoint: Epoch 20)

Metric	v1	v2 (ep 8)	v3
EN→DE BLEU (test)	5.37	≈ 0	0.01
DE→EN BLEU (test)	3.47	≈ 0	0.00
Average BLEU (test)	4.42	≈ 0	0.01
Best val loss	17.08	14.18	12.61
Tokenisation	word-10k	BPE-8k	BPE-8k
Attention	none	dot	dot
Encoder layers	1	1	2
Cycle warmup	—	—	yes
Learning rate	3e-4	3e-4	1e-3

3.4 Positive Signal: Val Loss Improvement

Despite near-zero BLEU, the val loss progression is meaningful:

	Val Loss ep 1	Val Loss ep 20
v1	32.51	17.08 (ep 15)
v2	14.84	14.18 (ep 8, terminated)
v3	14.27	12.61

The monotonic decrease in v3 (14.27 → 12.61, a 11.6% reduction) and the steady NMT loss decline

($17.55 \rightarrow 12.15$) confirm that the model's internal representations are improving consistently. The problem is not that the model fails to learn — it is that greedy decoding fails to exploit what it has learned.

3.5 Diagnosis: Why v3 BLEU Remains Zero

3.5.1 Decoder Fixed-Point Collapse (Confirmed by Probing)

After training, the decoder was probed step-by-step on the sentence "the house is on the left side of the street":

Step	Token	Top-5 (token, prob)
1	`die`	`die` 0.087, `das` 0.086, `der` 0.061, `ist` 0.029, `in` 0.029
2	`die`	`die` 0.061, `.` 0.043, `der` 0.034, `das` 0.028, `ist` 0.022
3	`die`	`die` 0.054, `.` 0.051, `der` 0.034, `das` 0.025, `ist` 0.020
4–10	`die`	*identical distribution to step 3*

Steps 3 through 10 produce **identical token distributions**. The decoder has entered a fixed point. The mechanism is clear:

$$h_2 = f(h_1,\ \mathrm{Embed}(\texttt{die}) + \mathrm{ctx}_1) \approx h_1 \tag{10}$$

$$\mathrm{ctx}_2 \approx \mathrm{ctx}_1 \quad \Rightarrow \quad \mathrm{input}_3 \approx \mathrm{input}_2 \quad \Rightarrow \quad h_3 \approx h_2 \tag{11}$$

Input feeding is the mechanism causing the fixed point. Once "`die`" is fed back as input, the context vector stabilizes and every subsequent step receives nearly identical input, producing identical hidden states and identical output distributions.

3.5.2 Tied Embedding Lock (Confirmed by Flat Output Distribution)

The maximum output probability across all 20 epochs never exceeded 8.7% (observed at step 1 of the probe above). A flat near-uniform output distribution indicates that the tied output projection $\mathbf{W}_{\mathrm{out}} = \mathbf{E}_{\mathrm{tgt}}^{\top}$ cannot simultaneously satisfy:

1. Token embedding: mapping token IDs to meaningful dense vectors for the encoder/decoder input.
2. B-BP inversion: the embedding must be compatible with $\mathbf{U}^{\top}$ inversion.
3. Output scoring: projecting decoder hidden states to sharp token distributions.

The gradient competition from these three objectives prevents the output distribution from sharpening, causing argmax to always select the highest-frequency German article (`die`).

4 Planned Fixes for v4

Based on the precise diagnosis from the v3 probe, two surgical fixes are planned:

Component	v3	v4	Rationale
Output projection	Tied $\mathbf{E}_{\text{tgt}}^{\top}$	Separate $\texttt{Linear}(d, V)$ per language	Frees embedding from outpu
Input feeding	$\text{Embed}(y_{t-1}) + \text{ctx}_{t-1}$	$\text{Embed}(y_{t-1})$ only	Removes fixed-point mechan

These are minimal, surgical changes. The B-BP cell, stacked encoder, BPE tokenization, attention, and cycle warmup schedule are all preserved.

5 Cross-Experiment Summary

Feature	v1	v2	v3	v4 (planned)
B-BP cell ($\mathbf{U}$ / $\mathbf{U}^{\top}$)	✓	✓	✓	✓
Tokenisation	word-10k	BPE-8k	BPE-8k	BPE-8k
Attention	—	dot	dot	dot
Input feeding	—	✓	✓	**removed**
Tied output proj	✓	✓	✓	**untied**
Encoder layers	1	1	2	2
Cycle warmup	—	—	✓	✓
Learning rate	3e-4	3e-4	1e-3	1e-3
EN→DE BLEU	5.37	≈ 0	0.01	—
DE→EN BLEU	3.47	≈ 0	0.00	—
Best val loss	17.08	14.18	12.61	—

5.1 Interpretation for the Adigun-Kosko Framework

A key observation across all three experiments: **the B-BP cell itself is not the problem**. The single-matrix inversion mechanism ($\mathbf{U}^{\top}$) has been empirically validated in v1 (DE→EN BLEU reached 15.94 at epoch 6). Every failure in v2 and v3 traces to components surrounding the B-BP cell — tokenisation, the output projection, input feeding — not to the inversion mechanism itself.

The val loss trajectory confirms this: internal representations are improving (v1: 32.51→17.08, v2: 14.84→14.18, v3: 14.27→12.61). The B-BP encoder is learning. The breakdown is at the interface between the learned representation and the output vocabulary — a problem that untying the output projection in v4 is designed to solve.